

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
JANICA PRIMA GINTING
109082500064
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

SOAL LATIHAN

Statement Skema Pemrosesan Sekuensial

1.

Source Code:

```
package main

import (
    "fmt"
)

func main() {
    var dat, total, count float64

    fmt.Scan(&dat)
    for dat != 9999 {
        total += dat
        count++
        fmt.Scan(&dat)
    }

    if count == 0 {
        fmt.Println("Tidak ada data")
    } else {
        fmt.Printf("%.4f\n", total/count)
    }
}
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● reddemon@reddemon-PK: /media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soall/Soall.go
10.5
20.5
30.0
9999
20.3333
○ reddemon@reddemon-PK: /media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$
```

Deskripsi Program:

Program ini digunakan untuk menghitung nilai rata-rata dari kumpulan bilangan desimal yang dimasukkan oleh pengguna. Cara kerjanya menggunakan sistem penanda, di mana program akan terus meminta input data dan baru akan berhenti berputar ketika pengguna memasukkan angka 9999. Setelah selesai, program langsung membagi total nilai dengan jumlah data yang masuk untuk mencetak hasil rata-ratanya, serta sudah dilengkapi pengondisian aman jika pengguna langsung memasukkan angka penanda di awal tanpa menginput data nilai sama sekali.

2.

Source Code:

```
package main

import (
    "fmt"
)

func main() {
    var x string
    var n int

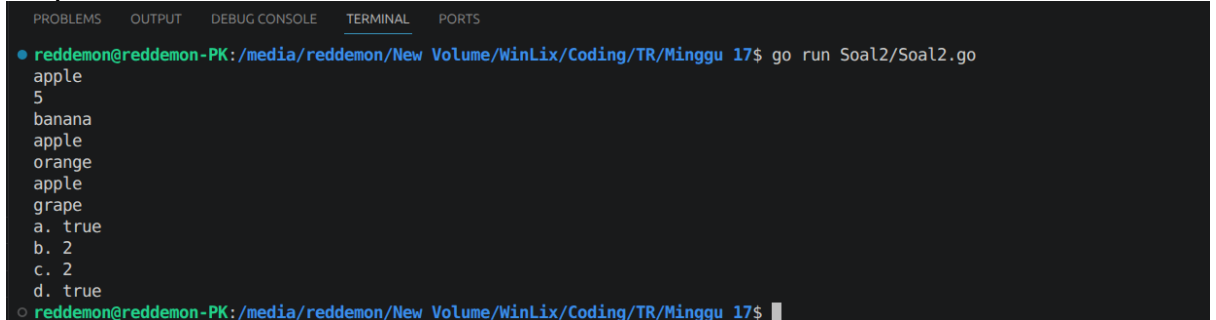
    fmt.Scan(&x)
    fmt.Scan(&n)

    found := false
    pos := -1
    count := 0

    for i := 1; i <= n; i++ {
        var current string
        fmt.Scan(&current)
        if current == x {
            if !found {
                found = true
                pos = i
            }
            count++
        }
    }

    fmt.Printf("a. %t\n", found)
    fmt.Printf("b. %d\n", pos)
    fmt.Printf("c. %d\n", count)
    fmt.Printf("d. %t\n", count >= 2)
}
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soal2/Soal2.go
apple
5
banana
apple
orange
apple
grape
a. true
b. 2
c. 2
d. true
○ reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$
```

Deskripsi Program:

Program ini digunakan untuk mencari kemunculan suatu kata kunci di dalam kumpulan data sebanyak n kata yang dimasukkan oleh pengguna. Cara kerjanya adalah dengan memeriksa setiap kata satu per satu menggunakan perulangan, lalu mencatat hasilnya untuk menjawab empat hal secara sekaligus: apakah kata tersebut ada atau tidak, di posisi keberapa kata itu pertama kali ditemukan, berapa kali kata tersebut muncul, serta memastikan apakah kata itu muncul minimal dua kali atau lebih.

3.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var totalTetesan int
    fmt.Scan(&totalTetesan)

    rand.Seed(42)

    countA, countB, countC, countD := 0, 0, 0, 0

    for i := 0; i < totalTetesan; i++ {
        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5 && y < 0.5 {
            countA++
        } else if x >= 0.5 && y < 0.5 {
            countB++
        } else if x >= 0.5 && y >= 0.5 {
            countC++
        } else {
            countD++
        }
    }

    konversi := 0.0001
    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(countA)*konversi)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(countB)*konversi)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(countC)*konversi)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", float64(countD)*konversi)
}
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soal3/Soal3.go
10000000
Curah hujan daerah A: 250.0415 milimeter
Curah hujan daerah B: 250.0609 milimeter
Curah hujan daerah C: 249.9928 milimeter
Curah hujan daerah D: 249.9048 milimeter
○ reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$
```

Deskripsi Program:

Program ini digunakan untuk mensimulasikan pengukuran curah hujan di empat daerah berdasarkan sebaran acak tetesan air. Cara kerjanya adalah dengan membuat koordinat acak untuk setiap tetesan air yang masuk, lalu mengelompokkannya ke dalam empat wilayah kuadran yang berbeda melalui kondisi percabangan. Di akhir perulangan, program akan menghitung total tetesan di masing-masing daerah dan mengonversinya ke dalam satuan milimeter dengan mengalikan setiap tetesan bernilai 0.0001.

4.

Source Code :

```
package main

import (
    "fmt"
    "math"
)

func trunc10(val float64) float64 {
    return math.Trunc(val*1e10) / 1e10
}

func main() {
    var n int
    fmt.Print("N suku pertama: ")
    fmt.Scan(&n)

    var s float64 = 0
    var piCurrent, piNext float64
    i := 1

    for {
        pembagiCurrent := float64(2*i - 1)
        if i%2 == 1 {
            s += 1.0 / pembagiCurrent
        } else {
```

```

    s -= 1.0 / pembagiCurrent
}
piCurrent = s * 4

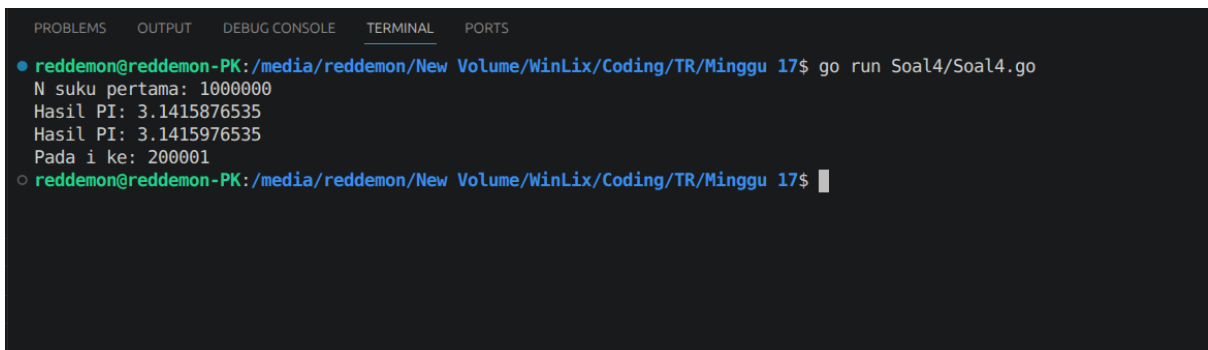
var sNext float64
pembagiNext := float64(2*(i+1) - 1)
if (i+1)%2 == 1 {
    sNext = 1.0 / pembagiNext
} else {
    sNext = -1.0 / pembagiNext
}
piNext = (s + sNext) * 4

if math.Abs(piCurrent-piNext) <= 0.00001 {
    i++
    break
}
i++
}

fmt.Printf("Hasil PI: %.10f\n", trunc10(piCurrent))
fmt.Printf("Hasil PI: %.10f\n", trunc10(piNext))
fmt.Printf("Pada i ke: %d\n", i)
}

```

Output :



```

reddeemon@reddeemon-PK: /media/reddeemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soal4/Soal4.go
N suku pertama: 1000000
Hasil PI: 3.1415876535
Hasil PI: 3.1415976535
Pada i ke: 200001
reddeemon@reddeemon-PK: /media/reddeemon/New Volume/WinLix/Coding/TR/Minggu 17$

```

Deskripsi Program :

program ini digunakan untuk menghitung estimasi nilai Pi menggunakan rumus deret harmonik Leibniz secara dinamis. Cara kerjanya adalah dengan membandingkan nilai hampiran Pi dari dua suku yang berurutan di dalam perulangan. Begitu selisih antara kedua nilai Pi tersebut sudah sangat kecil / kurang dari atau sama dengan 0.00001, perulangan akan langsung otomatis berhenti. Di akhir program, hasil cetak nilai Pi dipotong paksa pada digit ke-10 menggunakan fungsi kustom trunc10 agar nilainya tetap presisi dan akurat tanpa terkena pembulatan otomatis ke atas oleh sistem.

5.

Source Code :

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var totalTopping int
    fmt.Print("Banyak Topping: ")
    fmt.Scan(&totalTopping)

    rand.Seed(24)

    toppingDiPizza := 0

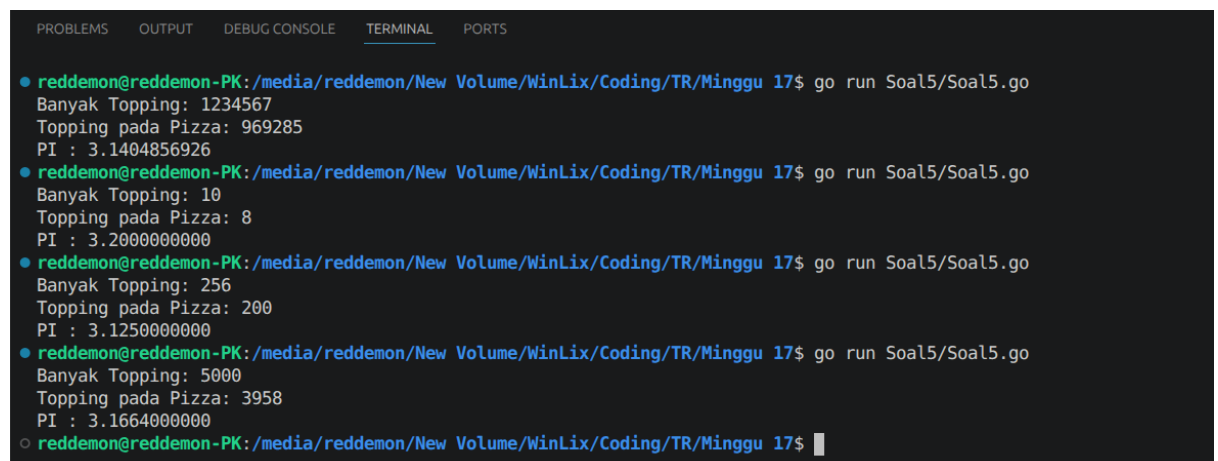
    for i := 0; i < totalTopping; i++ {
        x := rand.Float64()
        y := rand.Float64()

        dx := x - 0.5
        dy := y - 0.5

        if (dx * dx) + (dy * dy) <= 0.25 {
            toppingDiPizza++
        }
    }
    pi := 4.0 * float64(toppingDiPizza) / float64(totalTopping)

    fmt.Printf("Topping pada Pizza: %d\n", toppingDiPizza)
    fmt.Printf("PI : %.10f\n", pi)
}
```

Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• reddemon@reddemon-PK: /media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soal5/Soal5.go
Banyak Topping: 1234567
Topping pada Pizza: 969285
PI : 3.1404856926
• reddemon@reddemon-PK: /media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soal5/Soal5.go
Banyak Topping: 10
Topping pada Pizza: 8
PI : 3.2000000000
• reddemon@reddemon-PK: /media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soal5/Soal5.go
Banyak Topping: 256
Topping pada Pizza: 200
PI : 3.1250000000
• reddemon@reddemon-PK: /media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$ go run Soal5/Soal5.go
Banyak Topping: 5000
Topping pada Pizza: 3958
PI : 3.1664000000
○ reddemon@reddemon-PK: /media/reddemon/New Volume/WinLix/Coding/TR/Minggu 17$
```

Deskripsi Program :

program ini merupakan simulasi Monte Carlo untuk mengestimasi nilai π lewat perumpamaan menaburkan topping secara acak di atas pizza. Cara kerjanya adalah dengan membuat titik koordinat acak x, y untuk setiap topping, lalu memeriksa posisinya menggunakan rumus jarak atau persamaan lingkaran berpusat di $0.5, 0.5$ dengan jari-jari 0.5 . Jika hasil kuadrat jarak titik dari pusat kurang dari atau sama dengan 0.25 , berarti topping tersebut berhasil jatuh tepat di atas pizza bulat. Di akhir perulangan, nilai π dihitung secara matematis lewat perbandingan rasio antara jumlah topping yang masuk di area pizza dengan total keseluruhan topping yang disebar.